Skill Proof: An Immersive Simulation Platform for Demonstrating , Assessing Practical Skills Competency In Eswatini.

Section 1: Scope Alignment & Feature Validation

Planned Feature	Intended Purpose	Status	Evidence
Sign Up, Login, Logout, FireBase Authentication, storage	Link every result to a known user	Completed	Github Login, signup, firebase
User Profile Screen	Show user ID, email, average scores	Completed	Github User PS
Cloud Firestore data layer	Stores users, simulations and performance records	Completed	Github Cloud
Activity and trigger system	Track each step of a task	Completed	Github
Task progress screen and hints	Show live progress, errors and The next instruction	Completed	Github Hints
Electrical Wiring full Simulation (5 activities)	Show wiring competence and safety.	Completed	Github electric
Solar Panel full Simulation (5 activities)	Show solar panel installation competence and safety	Completed	Github Solar
Testable circuits breaker, sparks, test switches	Penalise wrong wiring and unsafe work, plus test option at the end.	Completed	Github Testable
Sound effects, correct sub activity sound, and complete activity sound, wrong activity sound, machine sounds	Give audio feedback,	Completed	Github Screenshort
Results Screen	Show complete time,errors, safety and competence, feedback	Completed	Github result
Evidence PDF and email	Give users proof they can keep and share	Completed	Github pdf
In app screen recording.	Record a session on headset , to share as proof	Completed	Github record
XR Keyboard	Lets user type in VR	Completed	Github Key
Tutorial Menu videos	Guides user on how to use system	Completed	Github Videos

Changes to original scope:
Firestore instead of Realtime Database: The data is structured by user and session, which handles well, and can login user past data from other devices.

Testing in Rwanda instead of Eswatini: Currently in Rwanda, so testing and data collection use participants here. The product is still designed for the Eswatini context.

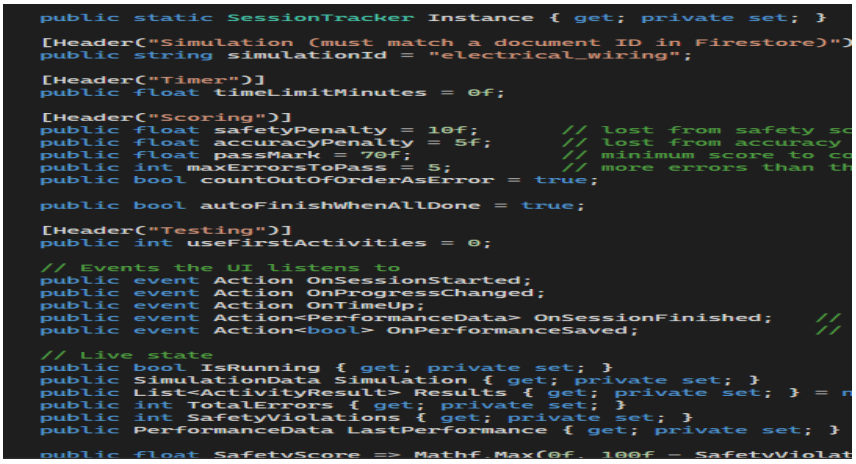
Evidence email sent: The feature to save evidence to a local device was changed to save evidence by sending the user an email with evidence which also allows the user to upload the file easier if the local device crushes.

Testing with a simulator and on the headset: The laptop does not meet the Quest link requirements so testing used the XR device Simulator and built and ran on the Quest 2.

Section 2: Technical Product Demonstration

A. Core Functionalities

Session tracker - The SessionTracker is the central script of the system. Every other script reports to it, and it is the only place where a session's results are calculated. When in the simulation it records five things, which activities are complete, procedural accuracy, errors, safety compliance and time used. When simulation ends, it turns these records into the safety, competency and procedural scores and results of competent or not competent.



```
public static SessionTracker Instance { get; private set; }

[Header("Simulation (must match a document ID in Firestore)")]
public string simulationId = "electrical_wiring";

[Header("Timer")]
public float timeLimitMinutes = 0f;

[Header("Scoring")]
public float safetyPenalty = 10f;       // lost from safety sc
public float accuracyPenalty = 5f;      // lost from accuracy
public float passMark = 70f;            // minimum score to ce
public int maxErrorsToPass = 5;         // more errors than th
public bool countOutOfOrderAsError = true;

public bool autoFinishWhenAllDone = true;

[Header("Testing")]
public int useFirstActivities = 0;

// Events the UI listens to
public event Action OnSessionStarted;
public event Action OnProgressChanged;
public event Action OnTimeUp;
public event Action<PerformanceData> OnSessionFinished;    //
public event Action<bool> OnPerformanceSaved;              //

// Live state
public bool IsRunning { get; private set; }
public SimulationData Simulation { get; private set; }
public List<ActivityResult> Results { get; private set; } = n
public int TotalErrors { get; private set; }
public int SafetyViolations { get; private set; }
public PerformanceData LastPerformance { get; private set; }

public float SafetyScore => Mathf.Max(0f, 100f - SafetyViolat
```

Activities and triggers - Each simulation has 5 activities and each activity has a side activity which will be counted to if each activity is completed. Triggers that share the same activityID are the tasks of one activity, if one of these activities with the same ID are not done then the full activity will not be complete until they are done. Socket tasks use an XR Socket interactor that accepts only the correct object.

Errors and safety - A wrong zone plays a warning sound first. The error is counted only if the object is still in the wrong place when the simulation ends. Testing a circuit the wrong way counts as an error at that moment. Starting work while the breaker is still ON is a safety violation, shown with sparks

and sound.

Scoring - The session tracker records completion, procedural accuracy, errors, safety compliance and time so from these it produces a safety, competency and procedural scores which then give a result of competent or not competent.

```
private PerformanceData BuildPerformance()
{
    int total = Results.Count;
    int completed = CompletedCount;
    bool allDone = completed == total;

    float safety = SafetyScore;
    float procedural = total == 0 ? 0f : (float)inOrderIds.Count / total * 100f;
    float accuracy = Mathf.Max(0f, 100f - TotalErrors * accuracyPenalty);
    float compliance = (safety + procedural) / 2f;
    float completion = total == 0 ? 0f : (float)completed / total * 100f;
    float competency = (completion + safety + procedural + accuracy) / 4f;

    bool competent = allDone
                  && safety >= passMark
                  && competency >= passMark
                  && TotalErrors <= maxErrorsToPass;

    string level = competency >= 90f ? "Advanced"
                 : competent ? "Competent"
                 : "Not Yet Competent";

    string feedback = competent
        ? "Well done. You completed every task safely and demonstrated competence."
        : allDone
            ? "Not yet competent. Reduce your errors and safety violations, then try again."
            : "Not yet competent. Some tasks were not completed. Review the steps you missed, then try again.";

    return new PerformanceData
    {
```

Evidence output - The EvidenceSaver script reads the results screen, builds a pdf with a small custom class (Evidence PDF), emails it through the Apps Script web app.

Scene management - The MenuNavigation script loads scenes in the background behind a loading screen. Every scene load must rest the scene and all activities. Only the logged-in user stays until logout.

Sound - A SoundManager with an audio mixer controls effects and ambience volume. It plays pickup, drop, machine, correct, wrong and complete sounds.

Skill Proofs runs on the Meta Quest 2 and talks to the Firebase directly.

Part	**Technology**	**Role**
VR client	Unity 6, C#, XR Interaction Toolkit	Scenes, interactions, activity tracking, scoring and user interface
Authentication	Firebase Authentication	Email and password sign-up and login. Firebase hashes the passwords.
Database	Cloud Firestore	Stores users, simulations and a performance record for every session
Email service	Google Apps Script web app	Receives the evidence and sends an HTML email to the user
Local storage	Quest device storage	Saves video evidence when recorded
Recording	Liv creators kit	Allows apps to have a recording feature.

B. Testing Approaches and Results

Three testing approaches were used.

Functional testing: each feature was tried with correct input and with wrong input, such as the wrong object in a socket, a test switch used too early and wiring started with the breaker ON.

Device and platform testing: features were first checked in the XR Device Simulator on the laptop, then built and run on the Meta Quest 2 to confirm they work on the real headset.

Error-handling testing: each mistake was checked to see that the right sound, effect and error or safety record was produced.

Table 1 shows the main test cases and their results.

Test	Input or action	Expected result	Result
Sign up	Valid name, email and password	Account is created, then the login screen opens	Pass
Login and logout	Valid account, then Logout	Simulation menu opens and profile shows name, ID and email; logout returns to login	Pass
Wrong object in socket	Place a wrong object in an XR Socket Interactor	Socket refuses it; the task stays incomplete	Pass
Remove a component	Complete an activity, then take a part out	Activity stops counting as complete	Pass
Test too early	Flip the test switch before all tasks are done	Sparks and wrong sound; one error is recorded	Pass
Breaker still ON	Start wiring with the breaker ON	Sparks and sound; safety violation recorded	Pass
Evidence email	Press Save Evidence on the results screen	PDF is built and the email arrives for each signed-in account	Pass
Scene restart	Restart the simulation, then use sockets	Scene and all activities reset and sockets work again	Pass

Results and defects found

Results and defects found. Some problems only appeared on the real headset or in real use. Each one was fixed:

Sockets stopped working after a restart. The scene was reloaded without resetting the activities. Every scene load now resets the scene and all activities, and only the logged-in user stays.
Password fields showed plain text on the Quest 2. The keyboard script now hides password input automatically on any headset.

The laptop could not run Quest Link. Testing was done in the XR Device Simulator and with Build And Run on the Quest 2.

The evidence email was blocked on the headset. The school account’s Apps Script deployment was locked to the school domain. It was redeployed from a personal Gmail account, and emails now arrive.

Activities stayed complete after a part was removed. An activity now stops counting as complete when a component is removed, and each socket accepts only the correct object.

Errors were counted as soon as an object entered a wrong zone. The zone now only plays a warning sound, and the error is counted at the end only if the object is still in the wrong place.
Profile data did not show right after login. A button in the simulation menu now loads the profile data immediately.

C. Code Quality Practices

One source of truth (Encapsulation, Singleton). SessionTracker is the only script that calculates results. Other scripts report to it through public methods, and it is reached through SessionTracker.Instance. This means scores cannot disagree between screens.

Reusable, inspector-driven scripts (DRY, Open/Closed). ActivityTrigger groups tasks by activityID, and ErrorTrigger and the test scripts take their settings in the Inspector. The same script works in every scene, and a new activity needs no code change.

Separate responsibilities (Single Responsibility, Separation of Concerns). Each script has one job: SoundManager for audio, MenuNavigation for scenes, EvidenceSaver for the PDF and email, and DatabaseManager for Firestore. The Scripts folder is split into Managers, Menu Navigation and Player.

Shared behaviour in one place (Abstraction, Observer). All sounds go through SoundManager and its audio mixer. Grab and drop sounds are written once and shared by every XR Grab Interactable through the selectEntered and selectExited events.

Safe handling of user data (Abstraction). Firebase Authentication handles and hashes passwords, so the app never stores them. The PDF is built by a small custom class with no extra libraries, which keeps the build light for the Quest 2.

Version control (Maintainability). The project is kept in a GitHub repository, and the README explains the setup and the project structure.